

DataBase-

Q1. What is indexing in databases?

Indexing in databases is a technique used to **speed up data retrieval** from a table.

Think of an index like the **index page of a book**:

- Without an index → you scan every page.
- With an index → you directly jump to the required page.

Similarly, a database index helps the DBMS quickly locate rows without scanning the entire table.

Example Without Index

Suppose you have a `users` table:

id	name	email
1	Amit	a@gmail.com
2	Rahul	r@gmail.com
3	Neha	n@gmail.com

Query:

```
SELECT * FROM users WHERE email = 'n@gmail.com';
```

Without indexing, the database performs a **full table scan**:

- Check row 1
- Check row 2
- Check row 3

This becomes slow for millions of rows.

Example With Index

Create index:

```
CREATE INDEX idx_email  
ON users(email);
```

Now the database can directly find the required row much faster.

Q2. How Index Works Internally

Most databases use a **B-Tree** data structure for indexes.

It stores:

- Indexed column values
- Pointers to actual rows

This allows:

- Fast searching
- Fast sorting
- Efficient filtering

Types of Indexes

1. Primary Index

Automatically created on a primary key.

```
CREATE TABLE employee(  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

Here, `id` gets indexed automatically.

2. Unique Index

Ensures values are unique.

```
CREATE UNIQUE INDEX idx_email  
ON users(email);
```

No duplicate emails allowed.

3. Composite Index

Index on multiple columns.

```
CREATE INDEX idx_name_city  
ON users(name, city);
```

Useful when queries use both columns together.

4. Clustered Index

Determines the physical order of data in the table.

- One clustered index per table
- Common in SQL Server

5. Non-Clustered Index

Separate structure pointing to actual data.

- Multiple allowed per table

Advantages of Indexing

- ✓ Faster `SELECT` queries
- ✓ Faster searching and sorting
- ✓ Improves `JOIN` performance
- ✓ Efficient filtering with `WHERE`

Disadvantages of Indexing

- ✗ Extra storage required
- ✗ Slower `INSERT`, `UPDATE`, `DELETE` because indexes also need updating

Q3. When to Use Indexing

Use indexes on:

- Frequently searched columns
- Columns used in:
 - `WHERE`
 - `JOIN`
 - `ORDER BY`
 - `GROUP BY`

Example:

```
SELECT * FROM orders
WHERE customer_id = 10
ORDER BY order_date;
```

Indexes on `customer_id` and `order_date` help.

Q4. What is Foreign Key in DB?

A **Foreign Key** is a column (or group of columns) in one table that creates a relationship with the **Primary Key** of another table.

It is used to maintain **referential integrity**, meaning:

- You cannot insert invalid data
- Relationships between tables remain consistent

SQL Example-

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);  
  
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    amount DECIMAL(10,2),  
  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)  
);
```

Q5. What Foreign Key Prevents

Suppose you try:

```
INSERT INTO orders VALUES (103, 10, 900);
```

If customer 10 does not exist in `customers`, the database throws an error.

This prevents invalid relationships.

Referential Integrity-

Foreign keys ensure:

- Child table values must exist in parent table
- No orphan records

Example:

- An order cannot exist without a valid customer

Q6. What are Foreign Key Actions?

1. CASCADE

If parent row is deleted, child rows also delete automatically.

```
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)
ON DELETE CASCADE
```

2. SET NULL

Child foreign key becomes NULL.

```
ON DELETE SET NULL
```

3. RESTRICT / NO ACTION

Prevents deleting parent row if child rows exist.

Q7. What is MySQL?

MySQL is an open-source relational database management system (RDBMS) that stores data in tables and uses SQL (Structured Query Language) for data manipulation.

Example - CREATE TABLE employee (

```
    id INT PRIMARY KEY,
    name VARCHAR(100),
    salary DECIMAL(10,2)
);
```

Q8. Primary Key vs Unique Key

Primary Key	Unique Key
Uniquely identifies row	Also ensures uniqueness
Only one per table	Multiple allowed
Cannot be NULL	Can contain NULL

Example - CREATE TABLE users(

id INT PRIMARY KEY,

email VARCHAR(100) UNIQUE

);

Q9. Clustered vs Non-Clustered Index

Clustered Index -

Data physically stored according to index.

In MySQL InnoDB: PRIMARY KEY is clustered.

Non-Clustered Index -

Stores pointer to actual data.

```
CREATE INDEX idx_name  
ON employee(name);
```

Question:

Why is primary key lookup faster?

Because InnoDB stores actual row data in clustered index.

Q10. Types of Joins

Suppose:

Employee

id	dept_id
1	10
2	20

Department

id	name
10	IT
20	HR

INNER JOIN

```
SELECT *  
FROM employee e  
INNER JOIN department d  
ON e.dept_id=d.id;
```

Returns matching rows.

LEFT JOIN

```
SELECT *  
FROM employee e  
LEFT JOIN department d  
ON e.dept_id=d.id;
```

Returns all employees.

RIGHT JOIN

Returns all departments.

SELF JOIN

```
SELECT e.name,  
       m.name manager  
FROM employee e  
JOIN employee m  
ON e.manager_id=m.id;
```

Q11. DELETE vs TRUNCATE vs DROP

DELETE	TRUNCATE	DROP
Removes rows	Removes all rows	Removes table
WHERE allowed	No WHERE	N/A
Rollback possible	Usually yes	No
Slower	Faster	Deletes structure

1.DELETE FROM employee WHERE id=1;

2.TRUNCATE TABLE employee;

3.DROP TABLE employee;

Q12. Why not normalize everything?

Too many joins reduce performance.

Use denormalization when needed.

Q13. What is a Transaction?

A transaction in MySQL is a group of SQL operations executed as a single unit of work. It ensures data integrity by following ACID properties. If all operations succeed, we use `COMMIT`; if any operation fails, we use `ROLLBACK` to undo all changes. A common example is transferring money between two bank accounts.

Example -

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1; -- Account A

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2; -- Account B

COMMIT;
```

```
START TRANSACTION; -- Begin transaction
```

```
COMMIT;           -- Save changes permanently
```

```
ROLLBACK;         -- Undo changes
```

```
SAVEPOINT sp1;    -- Create a checkpoint
```

```
ROLLBACK TO sp1;  -- Roll back to checkpoint
```


Q14. Do we have to type rollback or it is done automatically?

MySQL does not automatically execute `ROLLBACK` for statement errors inside a transaction. The application or user must call `ROLLBACK` explicitly. However, if a transaction is left uncommitted due to a crash or connection loss, InnoDB automatically rolls back the uncommitted changes during recovery. In frameworks like Spring Boot, rollback is often handled automatically through `@Transactional`.

Q15. ACID Properties

A – Atomicity

All or nothing.

C – Consistency

Data remains valid.

I – Isolation

Transactions don't interfere.

D – Durability

Committed data survives crashes.

Q16. What is a Deadlock?

Transaction A locks Row1 and wants Row2.

Transaction B locks Row2 and wants Row1.

Both wait forever.

MySQL detects and kills one transaction.

Q17. Difference Between WHERE and HAVING

WHERE -

Filters before grouping.

```
SELECT *  
FROM employee  
WHERE salary > 50000;
```

HAVING -

Filters after grouping. - The **HAVING** clause is used to **filter grouped data after aggregation** (COUNT, SUM, AVG, MAX, MIN).

```
SELECT dept_id,  
       COUNT(*)  
FROM employee  
GROUP BY dept_id  
HAVING COUNT(*) > 5;
```

Example 1: Customers whose total purchase is greater than 1000

```
SELECT customer_id, SUM(amount) AS total_amount  
FROM orders  
GROUP BY customer_id  
HAVING SUM(amount) > 1000;
```

Output:

customer_id	total_amount
-------------	--------------

101	1200
-----	------

103	1500
-----	------

Example 2: Departments with more than 5 employees

```
SELECT department_id, COUNT(*) AS employee_count  
FROM employees  
GROUP BY department_id  
HAVING COUNT(*) > 5;
```

Example 3: Using **WHERE** and **HAVING** together

Find customers whose orders are greater than ₹300 and whose total spending exceeds ₹1000:

```
SELECT customer_id, SUM(amount) AS total_amount  
FROM orders  
WHERE amount > 300  
GROUP BY customer_id  
HAVING SUM(amount) > 1000;
```

Q18. Subquery vs Join

Subquery-

```
SELECT *
FROM employee
WHERE dept_id IN (
    SELECT id
    FROM department
);
```

Join-

```
SELECT e.*
FROM employee e
JOIN department d
ON e.dept_id=d.id;
```

Generally joins perform better.

Q19. EXISTS vs IN

EXISTS -

Stops after first match.

```
SELECT *
FROM employee e
WHERE EXISTS (
    SELECT 1
    FROM department d
    WHERE d.id=e.dept_id
);
```

Good for large datasets.

IN –

Find employees who belong to IT or HR:

```
SELECT *
FROM employees
WHERE department IN ('IT', 'HR');
```

Q20. Find Second Highest Salary

Method 1-

```
SELECT MAX(salary)
FROM employee
WHERE salary <
(
    SELECT MAX(salary)
```

```
        FROM employee
    );
```

Method 2-

```
SELECT salary
FROM (
    SELECT salary,
           DENSE_RANK() OVER(
               ORDER BY salary DESC
           ) rnk
    FROM employee
) x
WHERE rnk=2;
```

The query uses the window function `DENSE_RANK()` to rank salaries in descending order. Employees with the same salary receive the same rank. The outer query filters rows with rank 2, which represents the second-highest distinct salary. `DENSE_RANK()` is preferred over `RANK()` when finding the Nth highest salary because it does not skip rank values when duplicates exist.

Q21. Difference Between CHAR and VARCHAR

CHAR-

Fixed length.

```
CHAR(10)
```

Always stores 10 chars.

VARCHAR-

Variable length.

```
VARCHAR(10)
```

Stores actual size.

Use VARCHAR most of the time.

Q22. What is a Composite Index?

```
CREATE INDEX idx_name_city
ON customer(name, city);
```

Works for:

```
WHERE name='John'
```

or

```
WHERE name='John'  
AND city='Pune'
```

Not efficient for:

```
WHERE city='Pune'
```

This is called the **Leftmost Prefix Rule**.

Q23. Top SQL Coding Questions

Find Duplicate Emails-

```
SELECT email,  
        COUNT(*)  
FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Find Nth Highest Salary-

```
SELECT salary  
FROM (  
    SELECT salary,  
           DENSE_RANK() OVER(  
               ORDER BY salary DESC  
           ) rnk  
    FROM employee  
) x  
WHERE rnk = N;
```

Employees Without Department-

```
SELECT e.*  
FROM employee e  
LEFT JOIN department d  
ON e.dept_id=d.id  
WHERE d.id IS NULL;
```

Q24. Order By

```
SELECT *  
FROM employee  
ORDER BY salary DESC;
```

Q25.